Solving Trajectory Optimisation: Shooting, iLQR and DDP

Contents

1. Overview . . . 2
2. Shooting: simulate the controls . . . 2
   - 2.1 The 8 s drive in numbers . . . 3
   - 2.2 Single shooting: simulate, measure the miss, adjust . . . 4
   - 2.3 Shooting the whole drive, and the long lever . . . 7
   - 2.4 Multiple shooting: shorter pieces, joined up . . . 9
3. Collocation on the same drive . . . 11
   - 3.1 Collocation checked on the straight drive . . . 11
   - 3.2 The whole drive . . . 12
   - 3.3 Which transcription fits a 0.1 s cycle . . . 14
4. iLQR and DDP: better steps from the model . . . 16
5. Direct and indirect methods . . . 19
6. Summary . . . 21
7. Sources . . . 22
8. Key terms . . . 22

Where this fits:

- **Builds on:** Hessian and multivariate Taylor; Gradient descent; Trajectory optimisation.
- **Compare with:** Direct collocation.

1. Overview

Key point: Shooting keeps only the controls and simulates the drive; collocation keeps the states too and ties neighbouring points by equations, so each of its equations touches only 12 numbers and every limit is a bound at a point, which suits a 0.1 s replanning cycle. iLQR solves a simpler version of the problem backwards along the drive and comes within 7 percent of the best cost in 13 iterations, where plain gradient descent is still at 174,704.

The robot can already reach the door past the person. Its local planner solves a trajectory optimisation problem over the next 1.5 m every 0.1 s and arrives at 8.0 s. Direct collocation (G-2893), which keeps the state and the controls at points in time and ties neighbours by the trapezoid rule, refined the whole A* route once into a 6.18 s drive.

Each solve must finish well inside the 0.1 s cycle: TEB's solver took 2.1 ms per cycle on average (Rösmann et al. 2013). Two choices decide whether a solver finishes in time:

- **how the problem is turned into numbers**, its transcription (G-2888): the local planner used collocation without comparing it with the other ways;
- **how each step is chosen:** plain gradient descent (G-862) on the controls of a whole 8 s drive to the door is still at a cost of 2,717.5 after 1000 iterations, against 0.217 for the best drive (notebook).

Which transcription fits a 0.1 s replanning cycle, and how can a solver take better steps? This Note answers both on one problem, the 8 s drive from the start to the door pose:

- shooting, which keeps only the controls and simulates the drive (Section 2);
- collocation on the same drive, and which of the three fits a 0.1 s cycle (Section 3);
- iLQR and DDP, better steps from the robot's model (Section 4);
- the other family of methods, which optimises before it discretises (Section 5).

2. Shooting: simulate the controls

Key point: Shooting keeps only the controls as unknowns and finds the drive by simulating it, then adjusts the controls until the drive ends at the goal. Single shooting is the simplest, but an early control moves the end of a long drive far more than a late one; multiple shooting cuts the drive into pieces to shorten that lever.

The opening's first choice is how to turn the problem into numbers. Collocation keeps the states as unknowns; this section tries the other way, keeping only the controls.

2.1 The 8 s drive in numbers

A whole drive to compare methods on. To compare methods on one problem we take the whole drive from the start to the door pose, without the person, in exactly 8 s with the least effort, a standard first example (Kelly 2017). The drive is long enough to show each method's weakness; refining once solved the same trip for the least time, 6.18 s.

The robot and its rules are those of the trajectory optimisation problem:

- x, y, the position of the robot's centre, in m, from (2.0, 1.0) at the start to (3.6, 3.5) at the door pose;
- θ, its heading, in rad, from 30° to 90°;
- v, its forward speed, in m/s, 0 at both ends;
- a, the acceleration, in m/s^2, a control, between -0.5 and 0.5;
- ω, the turn rate, in rad/s, the other control;
- the limits: each wheel at most 0.6 m/s, and the disk's centre at least 0.25 m from the table.

The cost, with:

- T, the duration of the drive, 8 s;
- J, the effort, in m^2/s^3: the acceleration squared plus 0.01 times the turn rate squared, added up over the drive, so that turning at 1 rad/s counts like speeding up at 0.1 m/s^2; squaring makes large commands cost much more than small ones, so the drive comes out smooth (Kelly 2017).

$$J = \int_0^T \left(a^2 + 0.01\,\omega^2\right) dt$$

The $\int$ sign adds up the quantity after it over every moment from 0 to T, the area under its curve (G-957).

The idea. A film is a list of still pictures, 24 every second, and the eye fills in between. A drive can be described the same way: cut the 8 s into 30 intervals of 0.267 s and hold the acceleration and the turn rate constant in each. The two functions become 30 pairs of numbers, 60 in all. Figure 1 shows the 60 numbers of the best drive: one step per interval in each control, and an acceleration that speeds up and then brakes.

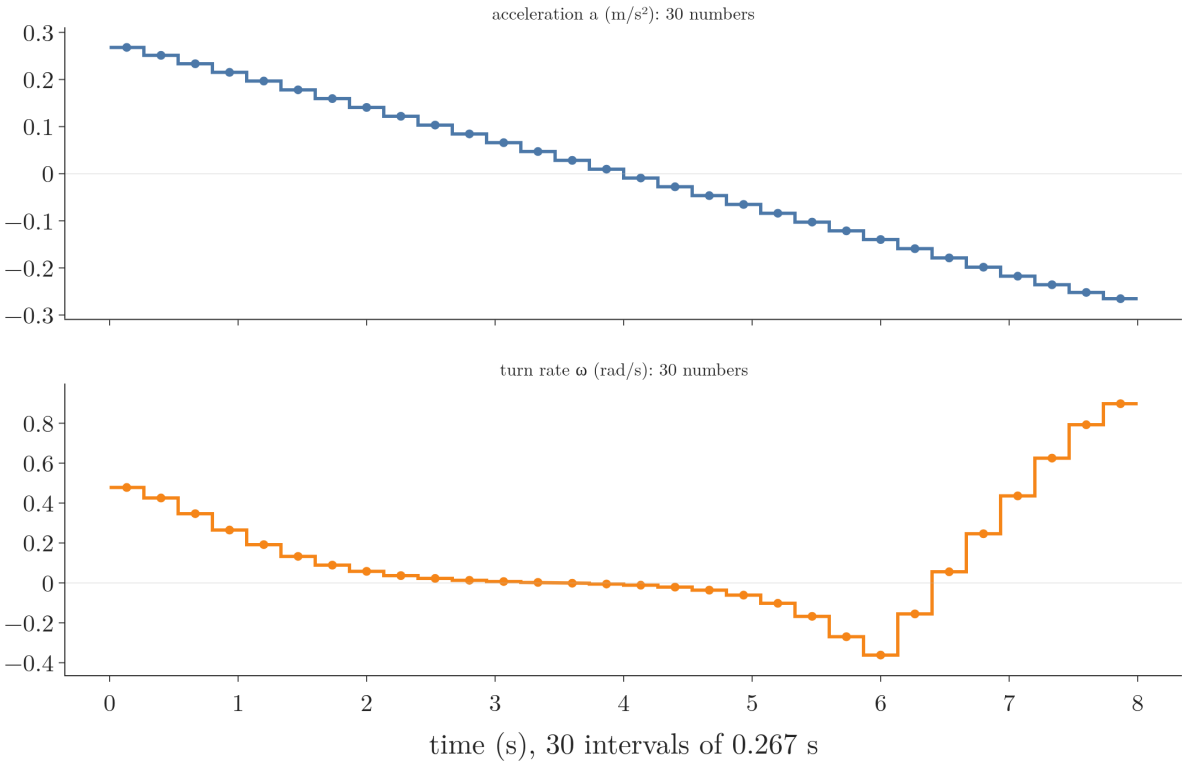


Figure 1: The controls of the 8 s least-effort drive (found in Section 2.3) as a list of numbers. The 8 s are cut into 30 intervals of 0.267 s, and in each the acceleration a (top) and the turn rate ω (bottom) are held constant: $30 + 30 = 60$ numbers. The acceleration falls in nearly a straight line from 0.27 to -0.27 m/s^2: speed up, then brake

Turning the functions of time into such a list is the drive's transcription (G-2888), the step that gives a solver numbers to change. Collocation keeps the states and the controls; keeping only the controls, and finding the states by simulating the motion rules, is shooting (Lynch and Park §10.7).

2.2 Single shooting: simulate, measure the miss, adjust

The idea. To hit a target with a cannon, fire, see where the ball lands, change the launch speed and fire again (Kelly, cannon). A drive works the same way: choose the controls, simulate the drive, and see where it ends.

We start with a straight drive with one number to choose, so the loop can be followed by hand. The robot must cover 3.0 m along a line in 10 s, from rest to rest: speed up at a for 5 s, then brake at a for 5 s. With:

- a, the acceleration, in m/s^2, the one number we choose (first guess 0);
- s_{end}, the distance covered after 10 s, in m;
- e, the miss, s_{end} minus the 3.0 m wanted, in m.

After 5 s the speed is 5 times a. The distance in each half follows from constant acceleration:

$$\text{first half} = a \times 5^2/2$$

$$= 12.5\,a$$

$$\text{second half} = 5a \times 5 - a \times 5^2/2$$

$$= 12.5\,a$$

$$s_{\text{end}} = 25\,a$$

The braking takes the speed from 5 times a back to 0, so every shot stops; only the distance can miss.

The miss and its slope. The first shot, $a = 0$, does not move: it misses by 3.0 m. To improve it we use gradient descent (G-862): take the squared miss as the cost and step against its slope. With:

- e, the miss, $25a - 3$, in m;
- C, the cost, the squared miss, e^2;
- η, the learning rate (G-1068), the size of each step, 0.0006.

The miss grows by 25 m for every 1 m/s^2 of a, so by the chain rule (G-371):

$$\frac{dC}{da} = 2e \times 25$$

$$= 50\,e$$

The first step, from $a = 0$ with $e = -3$, and the second shot:

$$\frac{dC}{da} = 50 \times (-3) = -150$$

$$a = 0 - 0.0006 \times (-150)$$

$$= 0.09$$

$$s_{\text{end}} = 25 \times 0.09 = 2.25 \text{ m}$$

$$e = 2.25 - 3 = -0.75 \text{ m}$$

Figure 2 fires the shots one per frame; watch the red miss shrink.

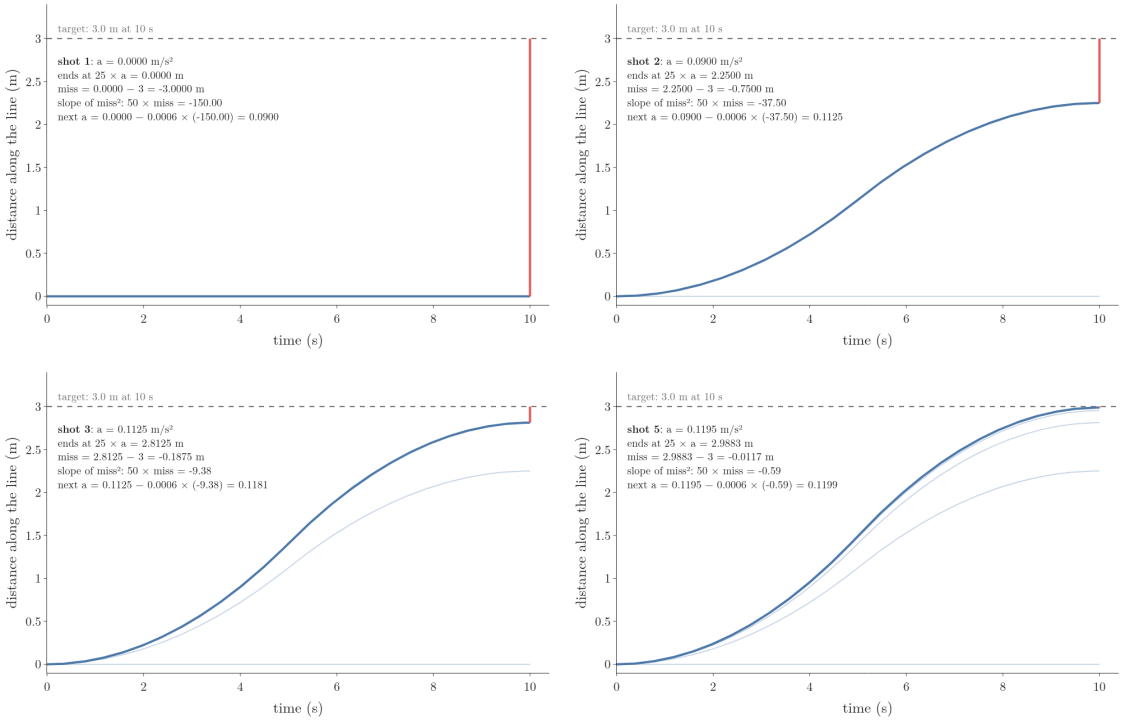


Figure 2: Single shooting with one number, the acceleration a, on the straight drive: speed up at a for 5 s, brake at a for 5 s, ending 25a metres along. Each frame fires one shot (blue: distance along the line over time; earlier shots pale), measures the miss at 10 s against the target 3.0 m (dashed; the miss in red), and steps a against the slope of the squared miss, 50 times the miss, with step size 0.0006. a goes 0, 0.09, 0.1125, 0.1181, 0.1195; the miss 3.0, 0.75, 0.1875, 0.0469, 0.0117 m

Each step cuts the miss to a quarter: 3.0, 0.75, 0.1875, 0.0469 m (notebook). The quarter follows from one step, with e_{new} the miss after it:

$$e_{\text{new}} = 25\,(a - 0.0006 \times 50\,e) - 3$$

$$= (25a - 3) - 0.75\,e$$

$$= e - 0.75\,e$$

$$= 0.25\,e$$

The shots converge on the a that makes the miss 0:

$$a = 3/25 = 0.12 \text{ m/s}^2$$

which reaches 0.6 m/s at 5 s, the robot's top speed.

The standard terms. Keeping only the controls as unknowns, finding the trajectory by one simulation from the start, and adjusting the controls until the end meets the goal is **single shooting** (G-2890), also called direct shooting (Kelly 2017; Kelly, cannon). The gap between where a simulation ends and where it must end is a **defect** (G-2891) (Kelly, cannon); the solver drives every defect to 0.

2.3 Shooting the whole drive, and the long lever

The whole drive has 60 numbers instead of one, and turning makes the motion rules curved. The loop is the same:

1. guess the 60 controls; our first shot faces the goal and speeds up and brakes too gently, falling 1.048 m short of the goal (notebook);
2. simulate the drive from the start, in steps of 0.067 s, with a fourth-order Runge–Kutta step, a more accurate Euler step (G-2299) that samples the motion rules four times per step (Kelly 2017);
3. measure the miss at the end and how far the drive breaks a limit or enters the table's band;
4. change all 60 controls at once, using the Jacobian of every miss and broken limit, with SciPy's least-squares solver, which takes Gauss–Newton steps like the timed elastic band's solver (Virtanen et al. 2020);
5. repeat until the miss is 0 and no limit is broken.

Misses and broken limits are added to the cost as penalties whose weight rises from 100 to 1,000,000, so that by the end they must be 0 (Tedrake, Underactuated ch. 10). Figure 3 runs it; watch the end of the blue drive converge on the door pose.

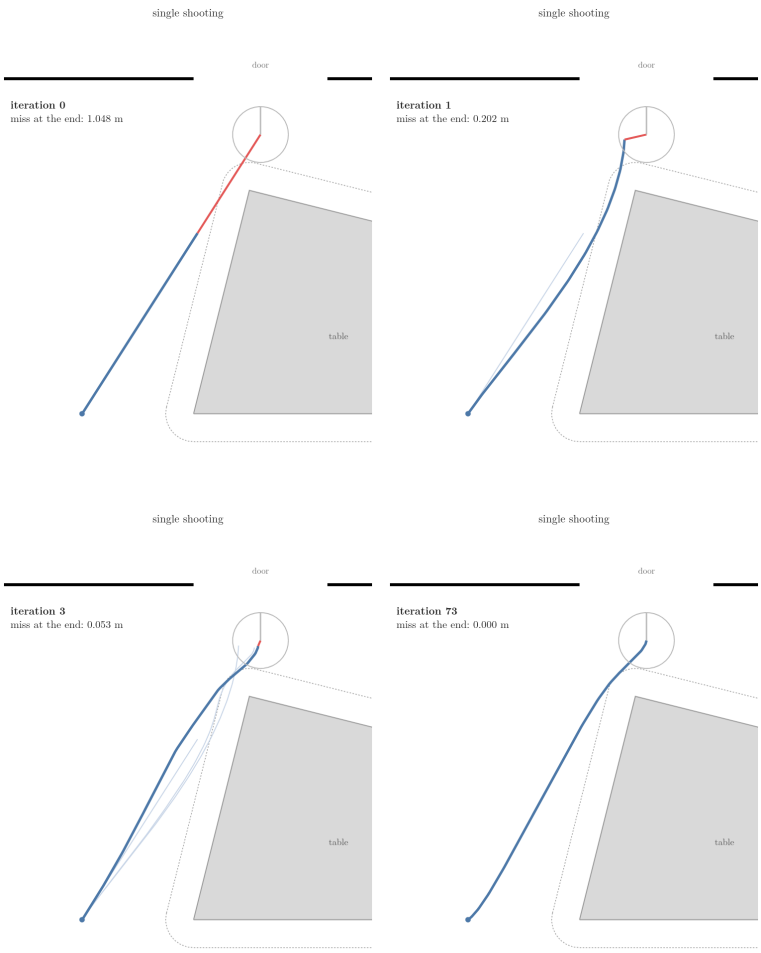


Figure 3: Single shooting of the 8 s least-effort drive to the door pose, one solver iteration per frame (thinning out after the first 12). Each frame is one simulation from the start with the current 60 controls (blue; the 4 before it pale), and the red line is the miss at the end. The first shot, facing the goal and too gentle, ends 1.048 m short; early iterations cut through the table's band while the penalty weight is low; after 73 iterations the drive ends on the door pose and passes the table's corner

After 73 iterations the drive ends on the door pose, touches the table's band to within 1 mm, and costs an effort of 0.217 (notebook).

The long lever. A control early in the drive acts on everything after it; a late control acts on little. On the straight drive with two numbers, with:

- a_1, the acceleration for the first 5 s, in m/s^2;
- a_2, the acceleration for the last 5 s, in m/s^2.

The first half covers 12.5 times a_1 metres, as in Section 2.2, and ends at 5 times a_1 m/s, a speed the second half keeps for 5 s more:

$$s_{\text{end}} = 12.5\,a_1 + 5a_1 \times 5 + 12.5\,a_2$$

$$= 37.5\,a_1 + 12.5\,a_2$$

The first number moves the end 3 times as far as the second. On the whole drive, 0.1 rad/s more turn rate in the first interval moves the end 0.079 m, and the same change in the last interval about 0.0001 m (notebook). The early control's larger effect is real, not a flaw of the method: it has more time to act (Tedrake, Underactuated ch. 10). But the solver must balance numbers whose effects differ by orders of magnitude, a problem called ill-conditioned, the property that a condition number (G-441) measures: small errors in a step are magnified, so on long or sensitive problems its steps become unreliable (Tedrake, Underactuated ch. 10). Limits along the way are also hard to impose, because each is a long chain of simulation back to the controls (Kelly 2017). The fix is to shorten the lever.

2.4 Multiple shooting: shorter pieces, joined up

The idea. Instead of one long shot, fire several short ones. Cut the 8 s into 5 pieces of 1.6 s. Guess the robot's full state at the start of each piece, and simulate each piece from there with its own 6 intervals of controls. At first the pieces do not meet; the solver moves both the controls and the start states until each piece ends where the next one starts (Kelly 2017; Kelly, cannon). Our first guess puts the start states on the A* route at a steady speed. Figure 4 runs it; watch the red gaps between the coloured pieces close.

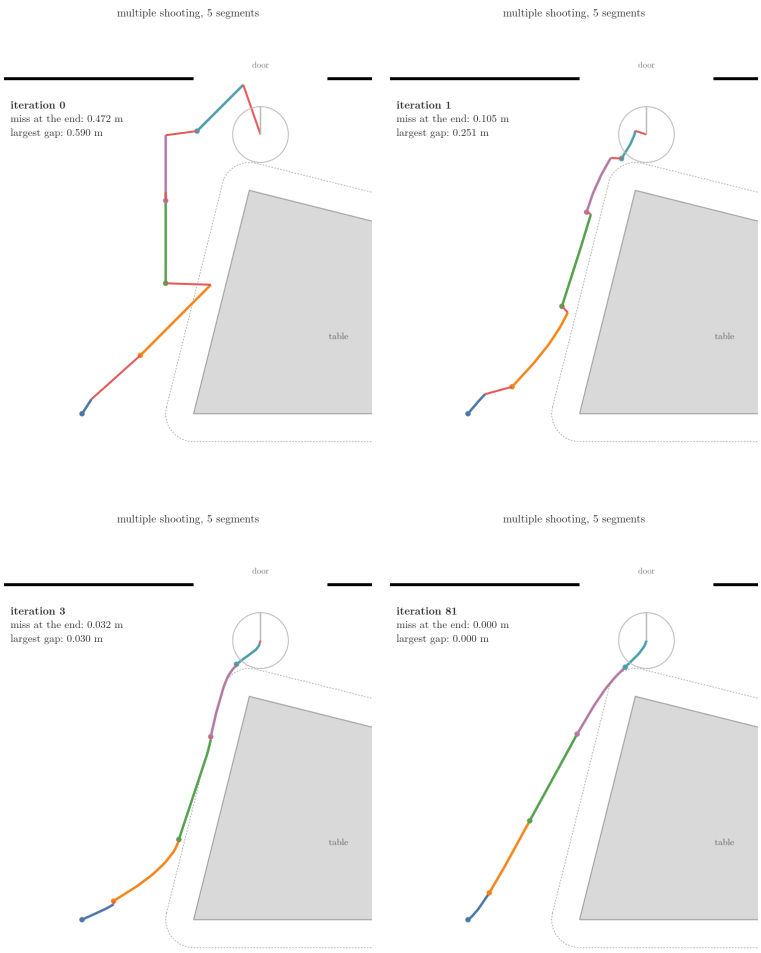


Figure 4: Multiple shooting of the 8 s least-effort drive, one solver iteration per frame (thinning out after the first 12). The drive is cut into 5 pieces of 1.6 s (colours); each is simulated from its own start state (dot), an unknown, first placed on the A* route. Red lines are the defects: the gap between a piece's end and the next piece's start, and the miss at the door pose. The largest first gap is 0.590 m; after 81 iterations every gap is 0 and the drive is the one single shooting found

The standard term. Shooting with the trajectory cut into segments, each simulated from its own start state, which is an unknown, with a defect at every join constrained to 0, is **multiple shooting** (G-2892), also called parallel shooting (Kelly 2017).

The unknowns are now the 60 controls plus 4 start states of 4 numbers, 76 in all. In exchange:

- each control acts only within its piece: 0.1 rad/s more turn rate in the first interval moves the end of its 1.6 s piece by 0.008 m, against 0.079 m for the whole drive (notebook);
- the pieces can be simulated in parallel (Kelly, cannon);
- multiple shooting converges on hard problems where single shooting fails (Kelly 2017;

Kelly, cannon).

On our drive both converge: multiple shooting takes 81 iterations to single shooting's 73, from first gaps of up to 0.590 m, and ends on the same drive with the same effort, 0.217 (notebook). Shorter pieces mean shorter levers; at the limit, every interval is its own piece and the state at every point is an unknown. Then nothing needs to be simulated at all.

3. Collocation on the same drive

> **Key point:** At one interval per piece, multiple shooting becomes direct collocation, and nothing is simulated. On the 8 s drive all three transcriptions find the same drive in 72 to 81 iterations, but each collocation constraint touches only 12 of its 186 numbers and every limit is a bound at a point, so collocation suits a 0.1 s replanning cycle.

Section 2.4 left the pieces of multiple shooting short, and the opening's question, which transcription fits a 0.1 s cycle, needs the third way too: at one interval per piece every state is an unknown, and the motion rules can be written as equations instead of being simulated (Tedrake, Underactuated ch. 10). The result is direct collocation (G-2893): the state and the controls at the knot points (G-2895), points in time, tied between neighbours by the trapezoid rule, the collocation constraint (G-2894).

3.1 Collocation checked on the straight drive

Before the whole drive, a check where the answer is known. For the straight 3.0 m drive of Section 2.2, now in the least time, the answer is: speed up at 0.5 m/s^2 for 1.2 s to 0.6 m/s, cruise, brake, 6.2 s (the fastest straight drive). Trapezoidal collocation with the time as an unknown gives (notebook):

Intervals	Least time found (s)
10	6.364
20	6.239
40	6.209

Figure 5 shows why it is not exact: the speed can bend only at knot points, so the sharp corners where the robot stops speeding up are cut. More knot points sharpen them, and the time approaches 6.2 s (Kelly 2017).

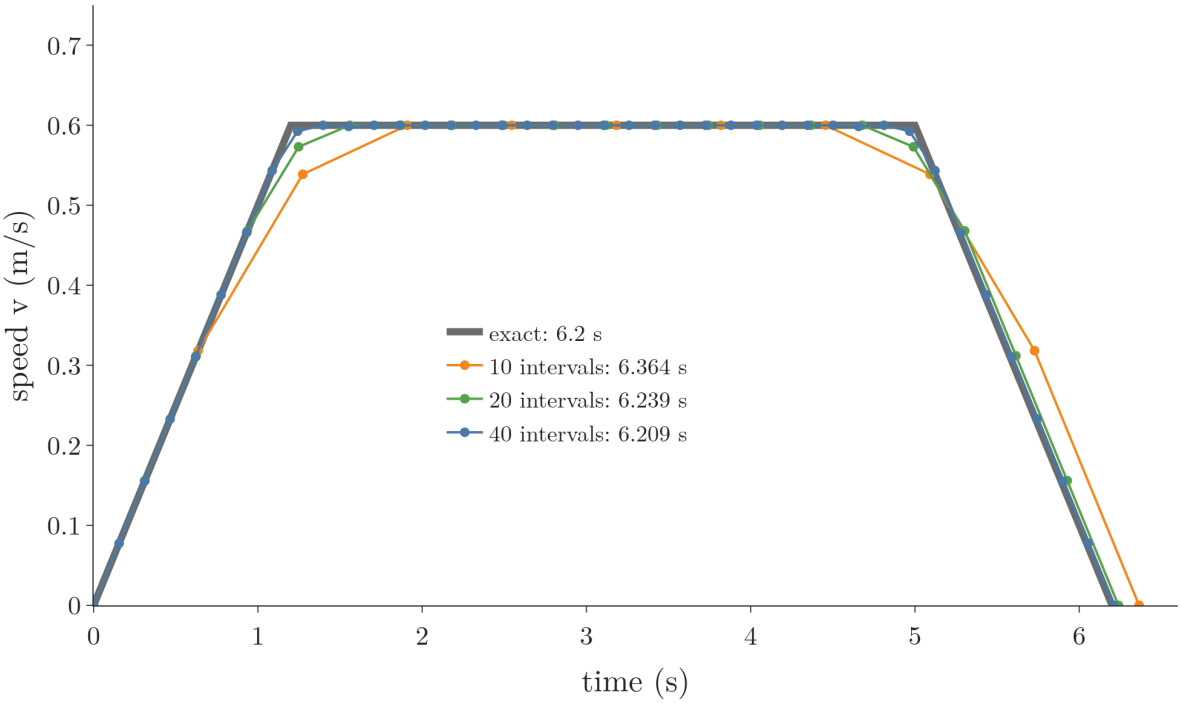


Figure 5: Collocation checked on the straight drive, whose least time is known: 6.2 s (grey: up at 0.5 m/s^2 to 0.6 m/s, cruise, brake). The speed at the knot points of the least-time collocation with 10 (orange), 20 (green) and 40 (blue) intervals. The corners are cut where the knot points are far apart; the times found, 6.364, 6.239 and 6.209 s, approach 6.2 s as the intervals shrink

Accuracy costs size: each knot point here holds 3 unknowns, the distance, the speed and the acceleration, and the time is one more:

$$10 \text{ intervals} = 11 \times 3 + 1 = 34 \text{ unknowns}$$

$$40 \text{ intervals} = 41 \times 3 + 1 = 124 \text{ unknowns}$$

3.2 The whole drive

The first guess puts the 31 knot points of the 8 s drive on the A* route at a steady speed, with the heading turning evenly from 30° to 90°. The guess breaks the motion rules: its largest defect, the gap the trapezoid rule leaves between where a knot point is and where the rule from its neighbour puts it, is 0.218 m (notebook). Figure 6 runs the same least-squares solver as in Section 2.3; watch the knot points slide off the route's corners onto a smooth curve past the table.

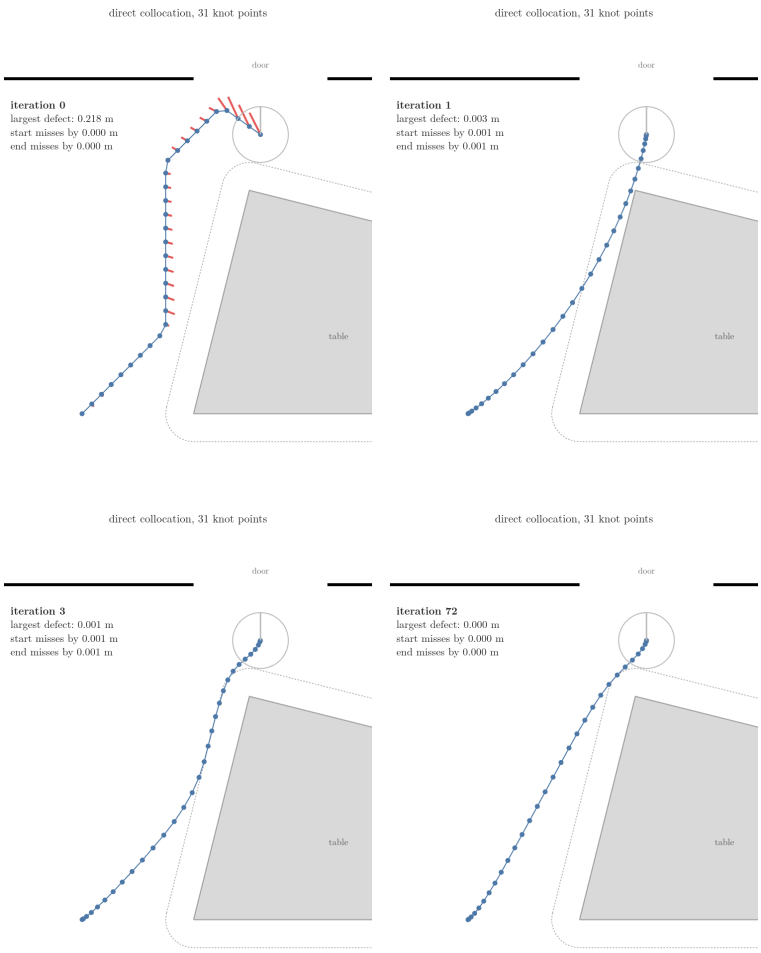


Figure 6: Direct collocation of the 8 s least-effort drive, one solver iteration per frame (thinning out after the first 12). The unknowns are the state and the controls at 31 knot points (dots), 0.267 s apart; nothing is simulated. The first guess puts them on the A* route, with the heading turning evenly from 30° to 90°. Red lines show each knot point's defect, the gap between where the trapezoid rule from the previous knot puts it and where it is: the largest is 0.218 m at first; early iterations cut through the table's band while the penalty weight is low; after 72 iterations every defect is 0

After 72 iterations every defect is 0 and the drive matches the shooting one: effort 0.218 against 0.217 (notebook). The small difference comes from the two transcriptions: collocation lets the controls change in a straight line between knot points, shooting holds them constant over each interval. Limits are simple, because the states are unknowns: the wheels, the speed and the table are checked at every knot point and halfway between, with no simulation in between (Kelly 2017).

3.3 Which transcription fits a 0.1 s cycle

The opening asked which transcription, which way of turning the problem into numbers, fits a 0.1 s replanning cycle. On the 8 s drive the three agree on the drive and are close in iterations (notebook):

Transcription	Unknowns	Iterations	Effort
single shooting	60	73	0.217
multiple shooting	76	81	0.217
direct collocation	186	72	0.218

The iteration counts do not separate them; how their numbers are tied together does. The equations of the motion rules in each touch:

- single shooting: the 4 numbers of the miss at the end, which depend on all 60 controls;
- multiple shooting: a join's 4 numbers, which depend on its piece's 12 controls and on the start states of the two pieces it joins, 4 numbers each (the first piece starts at the fixed start, so its join touches 16);
- direct collocation: a collocation constraint's 4 numbers, which depend on its two knot points, 6 numbers each.

Counted:

$$\text{single shooting} = 60 \text{ of } 60$$

$$\text{multiple shooting} = 12 + 4 + 4$$

$$= 20 \text{ of } 76$$

$$\text{collocation} = 6 + 6$$

$$= 12 \text{ of } 186$$

Figure 7 marks, for each transcription, which unknowns each equation touches. Look at how the blue cells thin out from a full strip into a narrow band along the diagonal.

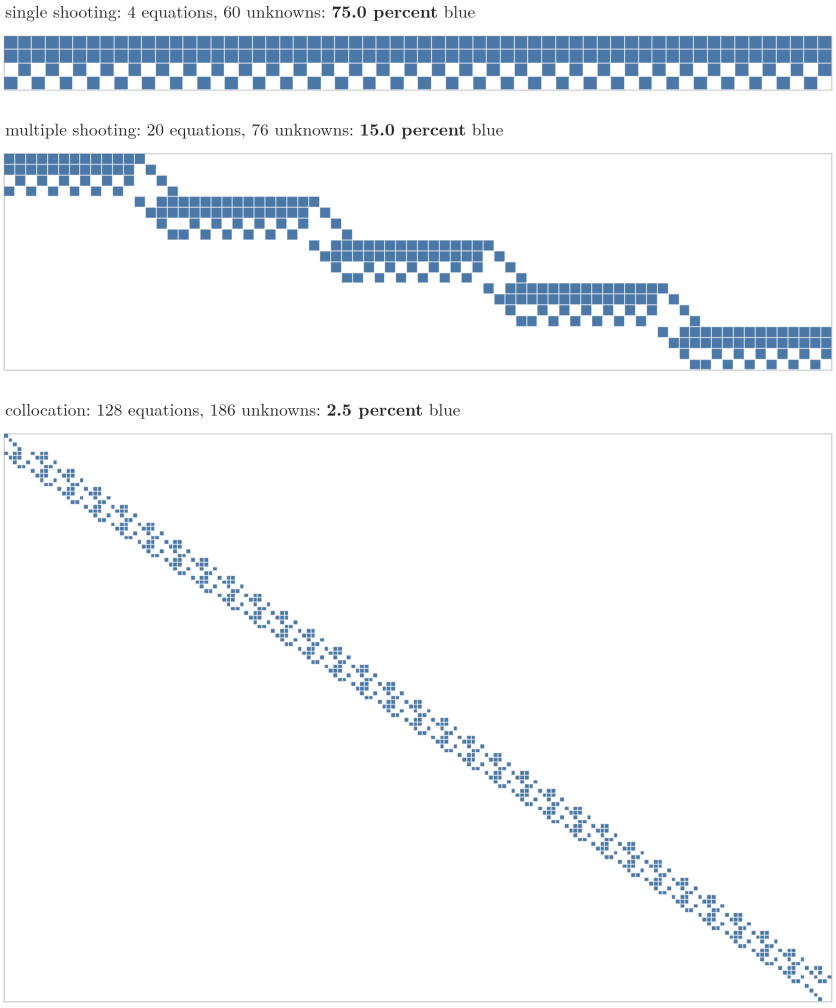


Figure 7: Which unknowns each equation of the motion rules touches, for the three transcriptions of the 8 s drive at their answers: one row per equation, one column per unknown, both in time order; blue where the equation changes when the unknown changes. Single shooting: the 4 numbers of the miss at the end against the 60 controls; the end heading depends only on the 30 turn rates and the end speed only on the 30 accelerations; 75 percent blue. Multiple shooting: the 4 joins and the end, 20 equations, against each piece's start state and its 12 controls; 15 percent. Collocation: the start, the 30 collocation constraints and the end, 128 equations, against the 6 numbers of each of the 31 knot points; 2.5 percent, in a band along the diagonal. Each panel is stretched to the same width, so its cells are smaller the more unknowns it has

Three things follow for a fast cycle:

- **The matrix is mostly zeros.** Collocation's equations touch only neighbours, so the matrix its solver works with is mostly zeros, with the same banded pattern in every such problem, and solvers are written to skip the zeros (Tedrake, Underactuated ch. 10); TEB's band is of this kind and took 2.1 ms per cycle (Rösmann et al. 2013). Shooting

gives a smaller matrix, but a full one, whose numbers are more coupled (Kelly 2017 §9.6).

- **Limits are bounds at points.** In collocation a limit at a moment is a bound on that knot point's numbers; in shooting it is a long chain of simulation back to the controls (Kelly 2017 §9.6).
- **No long lever.** Single shooting's first turn rate moves the end 0.079 m per 0.1 rad/s and its last one 0.0001 m; collocation spreads that effect over many equations of the same size (Tedrake, Underactuated ch. 10).

So collocation fits a local planner, whose problem has limits and obstacles all along its window and must be solved well inside 0.1 s. Kelly recommends collocation where the shape of the controls is not known in advance, such as a robot's motion, and multiple shooting over single shooting unless the controls are very simple or the first guess is very good (Kelly 2017 §9.11). TEB and Apollo's path optimiser both made that choice (TEB and Apollo's path QP are collocation). The transcription is chosen; each iteration still costs time, and the next section makes each iteration more effective.

4. iLQR and DDP: better steps from the model

> **Key point:** iLQR approximates the motion rules near the current drive by straight-line rules and the cost by a bowl, solves that easier problem exactly from the last step back to the first, then simulates the improved drive; repeated, it needs far fewer iterations than gradient descent. On the straight drive one iteration is exact; on the whole drive 13 iterations bring the cost from 1,422,936 to 0.233, where plain gradient descent is still at 174,704.

A local planner gets 0.1 s per cycle, so each solver step must be effective. Shooting in Section 2.3 needed a solver that aims its steps well; with plain gradient descent the steps are poor. On the whole drive's cost, gradient descent from the first shot is still at 2,717.5 after 1000 iterations, against 0.217 for the drive the other methods found (notebook). The robot's model gives more than a slope: how each step of the drive leads to the next.

The idea. Near the current drive, assume the motion rules are straight-line (linear) rules, which they nearly are for small changes, and that the cost is a bowl, a quadratic like the squared miss of Section 2.2. For that approximate problem the best change of the controls can be worked out exactly, from the end of the drive back to the start: at the last step, how should the control change, and how should it react if the robot is slightly off? Then at the step before, and so on back to the first. After this sweep back, simulate the drive forward with the new controls, correcting by the reactions where the robot drifts from the old drive. The new drive starts the next round (Tedrake, Underactuated ch. 10).

The standard terms. Repeating a sweep back that solves the linear-rules, bowl-shaped-cost version of the problem around the current trajectory, and a simulation forward with the result, is **iterative LQR** (G-2896), iLQR; LQR, the linear-quadratic regulator, is the exact method for linear rules and bowl-shaped costs that each sweep back applies (Tedrake, Underactuated ch. 10). **DDP** (G-2897), differential dynamic programming, also uses curved (second-order)

approximations of the motion rules; it is almost the same algorithm, and in most cases iLQR converges as quickly (Tedrake, Underactuated ch. 10). Both are close to shooting: they keep only the controls as unknowns and simulate the drive forward, but take each step by passing backward along the drive instead of using a general solver (Kelly 2017 §9.8). Each sweep back also gives the reactions, which say how to correct the controls when the robot drifts off the trajectory (Tedrake, Underactuated ch. 10).

A check at easy values. On the straight drive the motion rules are already linear and the effort is a bowl, so the approximate problem is the real one, and one iteration should land on the answer. We drive 3.0 m in 10 s with the least effort, the acceleration squared added up over the drive, the miss at the end costing 1,000,000 per m^2. The first guess, $a = 0$, misses by 3.0 m and costs 9,000,000; after one iteration the cost is 0.10812, and the second iteration changes nothing (notebook).

The exact answer is known: the least-effort move from rest to rest makes the acceleration fall in a straight line (Kelly 2017, §2.2, for a unit distance in unit time), scaled to any distance and time. With:

- ℓ, the distance, 3.0 m;
- T, the time, 10 s;
- $a(t)$, the acceleration at time t, in m/s^2.

$$a(t) = \frac{6\ell}{T^2}\left(1 - \frac{2t}{T}\right)$$

With our numbers, and the effort it costs:

$$a(t) = 0.18\,(1 - 0.2\,t)$$

$$\int_0^{10} a^2\,dt = 0.0324 \times \frac{10}{3}$$

$$= 0.108$$

The middle line uses the area under the squared bracket over the 10 s, a parabola:

$$\int_0^{10} (1 - 0.2\,t)^2\,dt = \frac{10}{3}$$

iLQR's 0.10812 differs from 0.108 only because it holds the acceleration constant over 30 intervals; its first interval holds 0.1742 m/s^2, the exact value at that interval's middle, 0.174 m/s^2 (notebook).

The whole drive. The real rules turn, so the approximate problem is only close, and iLQR needs several rounds. iLQR starts from the same first shot as single shooting, with the

effort plus penalties of weight 1,000,000 for missing the door pose, breaking a wheel limit or touching the table (iLQR handles constraints as penalties, Tedrake, Underactuated ch. 10). Figure 8 runs it next to plain gradient descent on the same cost.

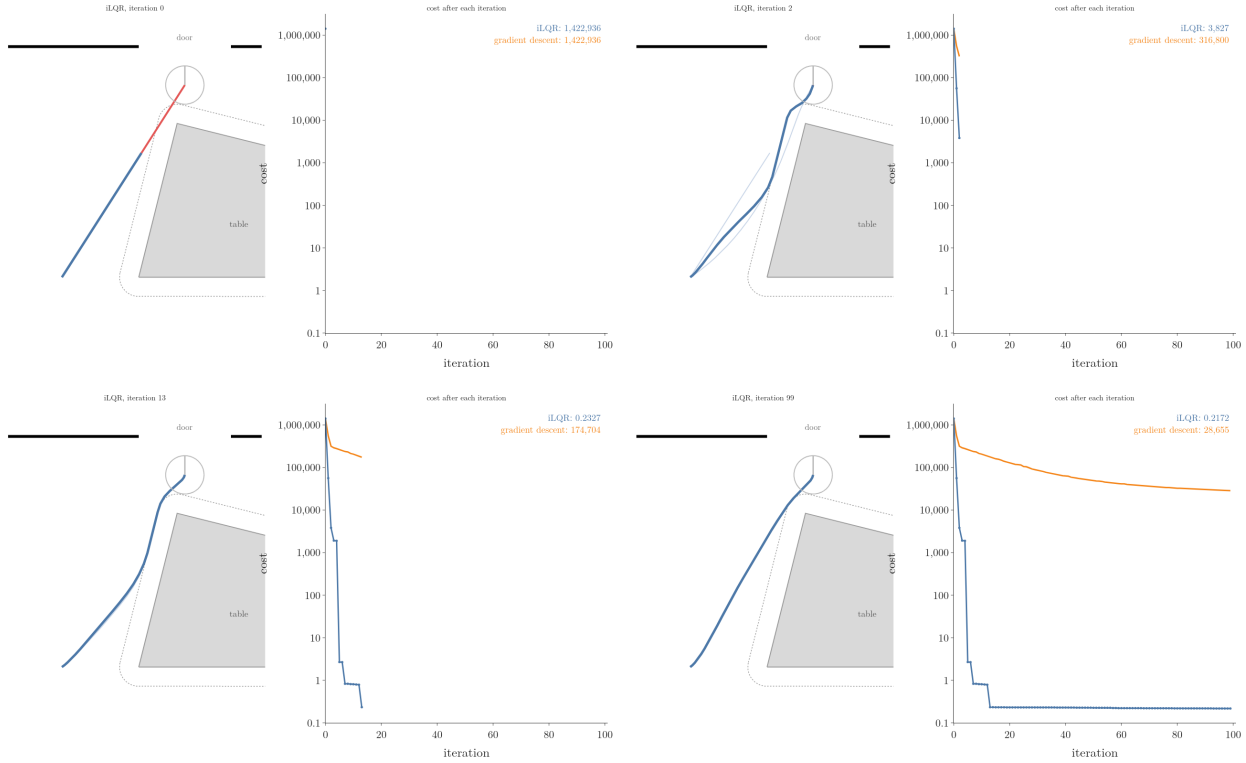


Figure 8: iLQR on the 8 s least-effort drive, one iteration per frame (thinning out after 15), from the same first shot as single shooting. Left: the drive after each iteration (blue; earlier ones pale) and the miss at the end (red). Right: the cost after each iteration, on a scale where each grid line is 10 times the one below it: iLQR (blue) against plain gradient descent on the same cost from the same start (orange). After 13 iterations iLQR is at 0.2327 and gradient descent at 174,704; after 99 iterations iLQR is at 0.2172, the drive the other methods found

After 13 iterations the cost falls from 1,422,936 to 0.2327, within 7 percent of iLQR's final 0.2172, reached at iteration 99 on the same drive as shooting and collocation (notebook). iLQR's steps use the curvature of the cost along the whole drive, as Newton's method (G-1321) uses the curvature of a function (Tedrake, Underactuated ch. 10). Shooting, collocation and iLQR all cut the problem into numbers first and then optimise. There is another approach.

5. Direct and indirect methods

> **Key point:** Every method so far discretises first and then optimises: a direct method. An indirect method first derives the conditions any optimal trajectory must meet, Pontryagin's minimum principle, and then solves them; for the straight drive with no speed limit they say full acceleration, then full braking, with one switch.

Every method so far, from the timed elastic band to iLQR in Section 4, turned the trajectory into a list of numbers before optimising it.

The standard terms. A method that discretises the trajectory optimisation problem into a list of numbers and then optimises them is a **direct method** (G-2898) (Kelly 2017). A method that first works out, by calculus, the conditions an optimal trajectory must meet, and then discretises and solves those conditions, is an **indirect method** (G-2899) (Kelly 2017). In short: a direct method discretises, then optimises; an indirect method optimises, then discretises (Figure 9).

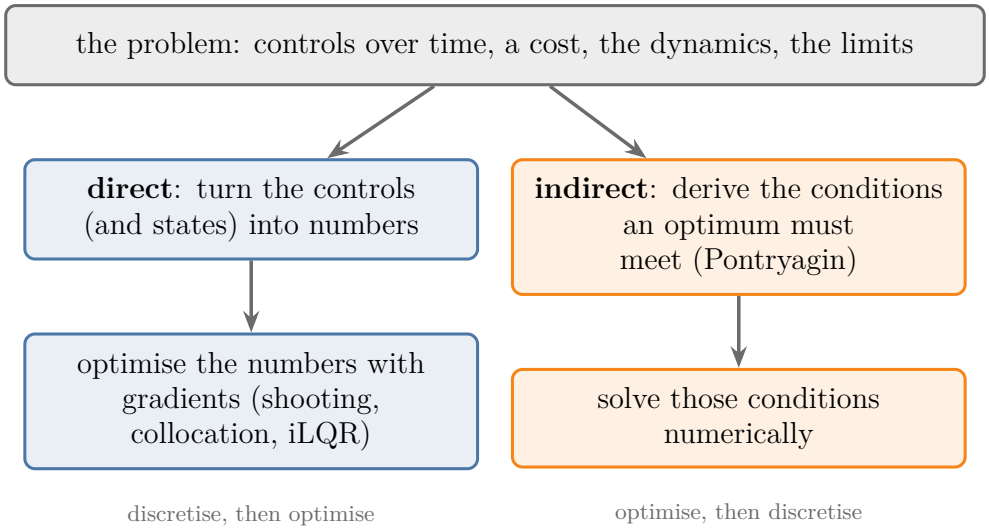


Figure 9: Two ways to solve a trajectory optimisation problem. Direct (blue): turn the controls, and maybe the states, into numbers, then optimise the numbers with gradients, as shooting, collocation and iLQR do: discretise, then optimise. Indirect (orange): derive the conditions an optimum must meet (Pontryagin's minimum principle), then solve those conditions numerically: optimise, then discretise

The conditions come from **Pontryagin's minimum principle** (G-2900): along an optimal trajectory, the controls at each moment minimise a certain function of the state, the controls and a second set of variables, the adjoint variables, that run backwards in time. The principle gives necessary conditions: every optimal trajectory meets them, but a trajectory that meets them need not be optimal (LaValle §15.2.3; Tedrake, Underactuated ch. 10).

On the straight drive. For a least-time drive whose only control is an acceleration between -0.5 and 0.5 m/s^2, with no speed limit, the principle says the acceleration is always at one

of its two limits and switches at most once: full acceleration to the middle of the 3.0 m, then full braking (LaValle §15.2.3, Example 15.4). Our robot also has its 0.6 m/s limit, so its fastest drive adds a cruise in between: the 6.2 s drive of Section 3.1. Speed profiles along a fixed path (RO-139), a later Note, teaches such drives.

The trade-off. An indirect method is generally more accurate, because its conditions are derived exactly, but (Kelly 2017):

- the conditions are hard to derive for most robots;
- it needs a first guess even for the adjoint variables, which have no physical meaning to guess from;
- it converges from a smaller range of first guesses.

A local planner meets a new problem every 0.1 s, with obstacles that change, so these costs weigh heavily: TEB, Autoware, Apollo and MoveIt, the real systems of the local planner, all use direct methods.

6. Summary

Idea	What it does	On the 8 s drive
single shooting	controls only, one simulation from the start	73 iterations; lever 0.079 m against 0.0001 m
multiple shooting	5 pieces, each from its own start state	81 iterations; lever 0.008 m
direct collocation	states and controls at knot points, nothing simulated	72 iterations; 12 of 186 numbers per equation
iLQR	steps from the model, backwards along the drive	13 iterations to within 7 percent
direct and indirect	discretise then optimise, or the reverse	TEB, Autoware, Apollo and MoveIt are direct

- Collocation fits a 0.1 s replanning cycle, because on the 8 s drive it finds the same drive as shooting (effort 0.218 against 0.217, in 72 iterations against 73 and 81) while each of its collocation constraints touches only 12 of its 186 numbers and every limit is a bound at a knot point, so the matrix of its motion-rule equations is 2.5 percent filled and fast to solve, as TEB's 2.1 ms cycles show.
- **Single shooting** (G-2890) simulates from the start and drives the **defect** (G-2891) at the end to 0, but its lever is long: an early turn-rate change moves the end 0.079 m, a late one 0.0001 m, so on long drives the problem is ill-conditioned and its steps become unreliable.
- **Multiple shooting** (G-2892) shortens the lever to 0.008 m by simulating 5 pieces from their own start states, so it converges on harder problems and its pieces can be simulated in parallel.
- Collocation is the limit of multiple shooting with one interval per piece, so nothing is simulated; its accuracy grows with the knot points, 6.364, 6.239 and 6.209 s against the exact 6.2 s, because the speed can bend only at them.
- **iLQR** (G-2896) and **DDP** (G-2897) solve a linear, bowl-shaped version of the problem backwards along the drive, so their steps are second order: 13 iterations to within 7 percent, where gradient descent is 750,000 times higher.
- **Direct methods** (G-2898) discretise and then optimise; **indirect methods** (G-2899) solve the necessary conditions of **Pontryagin's minimum principle** (G-2900), which are hard to derive and need a close first guess, so TEB, Autoware, Apollo and MoveIt all use direct methods.

The robot can now choose how its local planner turns the problem into numbers and how the solver steps. Still missing: the plan is played as commands over time, and a real robot slips and drifts away from it between plans; following a trajectory needs feedback, which measures where the robot is and corrects the commands, the subject of the next chapter, feedback control. iLQR's reactions already give such corrections near one trajectory.

7. Sources

Built from

- Matthew Kelly, "Introduction to Trajectory Optimization", a talk with slides given at Cornell University, YouTube, https://www.youtube.com/watch?v=wlkRYMVUZTs
- Kelly, M. (2017). "An introduction to trajectory optimization: how to do your own direct collocation". *SIAM Review* 59(4), 849–904, https://doi.org/10.1137/16M1062569, free PDF from the author: §2.2 (the least-effort block move), §9.4 to §9.6, §9.8 and §9.11 (indirect methods, single and multiple shooting, DDP, which method is best) (Kelly 2017)
- Kelly, M. "Cannon example: single shooting, multiple shooting, direct collocation", tutorial page, https://www.matthewpeterkelly.com/tutorials/trajectoryOptimization/canon.html (Kelly, cannon)
- Tedrake, R. *Underactuated Robotics*, ch. 10 "Trajectory optimization": direct shooting and numerical conditioning, sparsity of direct transcription, penalty methods, iterative LQR and DDP, Pontryagin's minimum principle, https://underactuated.mit.edu/trajopt.html (Tedrake, Underactuated ch. 10)
- LaValle, S. M. (2006). *Planning Algorithms*. Cambridge University Press. §15.2.3, Example 15.4, https://lavalle.pl/planning/ (LaValle §15.2.3)
- Lynch, K. M. and Park, F. C. (2017). *Modern Robotics: Mechanics, Planning, and Control*. Cambridge University Press. §10.7 "Nonlinear optimization", http://modernrobotics.org (Lynch and Park §10.7)

Other references

- Rösmann, C., Feiten, W., Wösch, T., Hoffmann, F. and Bertram, T. (2013). "Efficient trajectory optimization using a sparse model". *European Conference on Mobile Robots (ECMR)*, free PDF: the run time of TEB's solver (Rösmann et al. 2013)
- Virtanen, P. et al. (2020). "SciPy 1.0: fundamental algorithms for scientific computing in Python". *Nature Methods* 17, 261–272: the solver `least_squares` used in the notebook (Virtanen et al. 2020)

8. Key terms

Terms taught in this Note come first; linked terms are recaps, taught in the Note the link opens.

Term	Meaning
Single shooting (direct shooting) (G-2890)	Trajectory optimisation that keeps only the controls as unknowns, finds the trajectory by one simulation from the start, and adjusts the controls until the end meets the goal; simple, but an early control moves the end far more than a late one.

Term	Meaning
Defect (G-2891)	The gap between where a simulation or an integration rule says the trajectory should be and where the unknowns put it, such as a shot ending 1.048 m short of the door pose; the solver drives every defect to 0.
Multiple shooting (parallel shooting) (G-2892)	Shooting with the trajectory cut into segments, each simulated from its own start state, which is also an unknown, with a defect at every join constrained to 0; each control then acts on one short piece, which makes it more robust than single shooting.
iLQR (iterative LQR) (G-2896)	Trajectory optimisation that approximates the motion rules near the current trajectory by linear rules and the cost by a bowl, solves that version exactly in a sweep from the last step back to the first, and simulates the result forward; its steps are second order, so it needs far fewer iterations than gradient descent, and it also gives feedback gains.
DDP (differential dynamic programming) (G-2897)	iLQR with curved (second-order) approximations of the motion rules as well as of the cost; almost the same algorithm, and in most cases no faster to converge than iLQR.
Direct method (trajectory optimisation) (G-2898)	A method that first discretises the trajectory problem into a list of numbers and then optimises them, such as shooting, collocation, iLQR and TEB; easier to set up, and what real planners use.
Indirect method (trajectory optimisation) (G-2899)	A method that first derives by calculus the conditions an optimal trajectory must meet, then discretises and solves those conditions; more accurate, but the conditions are hard to derive and it needs a good first guess.
Pontryagin's minimum principle (G-2900)	The necessary conditions for an optimal trajectory: at each moment its controls minimise a function of the state, the controls and adjoint variables that run backwards in time; every optimum meets them, but meeting them does not prove optimality. Indirect methods solve them.

Term	Meaning
Direct collocation (G-2893)	Transcription in which the states and controls at a row of points in time are all unknowns and the motion rules are equations between neighbouring points; nothing is simulated, so limits along the way are easy to add.
Transcription (G-2888)	Turning a trajectory optimisation problem, whose unknowns are functions of time, into one whose unknowns are a finite list of numbers, such as a band's 10 poses and the 9 time steps between them, 39 numbers, so that an optimiser can search over them.
Gradient descent (G-862)	Finding the lowest point of a function by repeated small steps downhill.
Integration (G-957)	Finding the area under a curve by adding up infinitely many thin strips.
Learning rate (η) (G-1068)	How strongly each update changes the model; in gradient descent, the number the slope is multiplied by to get the step size.
Chain rule (G-371)	To differentiate a function of a function, multiply the outer derivative by the inner derivative.
Euler step (G-2299)	One step of predicting motion: hold the rates fixed for a short time Δt and add rate times Δt to each quantity; repeated, it traces a path, more accurately the smaller Δt is.
Condition number (G-441)	A number that says how much a matrix can magnify small errors, such as rounding errors, when we solve with it; a large value means the answer is sensitive. It is the largest singular value over the smallest, σ_1/σ_n.
Knot point (collocation point) (G-2895)	One of the points in time at which collocation keeps the state and controls as unknowns, such as 31 points 0.206 s apart over a 6.18 s drive; more knot points make the solution more accurate.
Collocation constraint (trapezoidal) (G-2894)	The equation that ties neighbouring knot points through the motion rules; in trapezoidal collocation the change in state equals the time step times the average of the rates at the two points, $x_{k+1} - x_k = \frac{h}{2}(f_k + f_{k+1})$, so a simulation is not needed.

Term	Meaning
Newton's method (G-1321)	A way to find a minimum: at the current point, fit a bowl (the second-order Taylor polynomial, from the first and second derivatives), jump to its lowest point, and repeat; the step is $\boldsymbol{\delta} = -H^{-1}\nabla f^{\mathsf{T}}$. XGBoost uses one such step for every tree.